

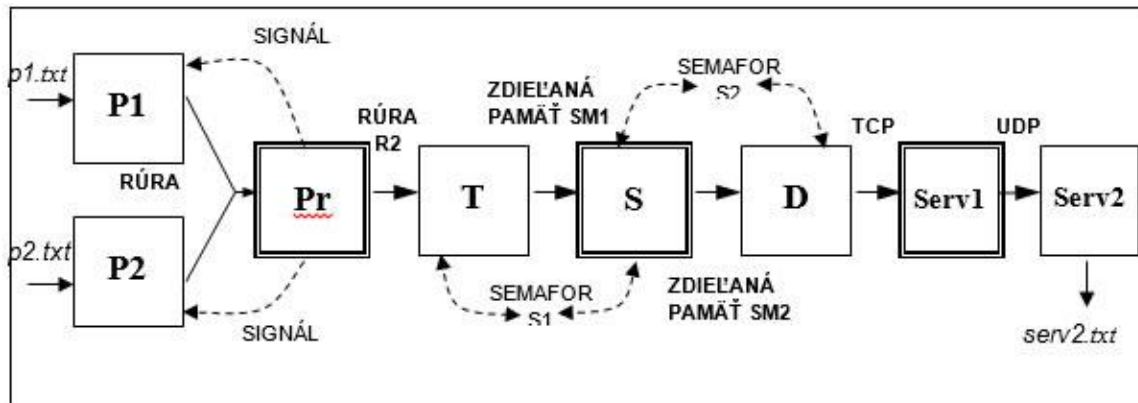
TECHNICKÁ UNIVERZITA V KOŠICIACH
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

OPERAČNÉ SYSTÉMY

Zadanie Unix1 – dokumentácia

1.TEXT ZADANIA

SCHÉMA



- procesy vyznačené zvýrazneným okrajom sú programy, ktoré budú pri kontrole zadania dodané. Teda treba vypracovať iba programy P1, P2, T, D a Serv2.

POPIS PROCESOV

PROCES Zadanie (vypracovaný študentom, nie je zakreslený v diagrame)

Spúšťanie

zadanie <číslo portu 1> <číslo portu 2>

Funkcia

Parametre hlavného programu sú čísla portov pre servery Serv1 (TCP) a Serv2 (UDP). Tieto čísla je potrebné týmto procesom odovzdať. Program **Zadanie** nech vyhradí všetky zdroje (pre medziprocesovú komunikáciu) a nech spustí všetky procesy. Všetky procesy nech sú realizované ako samostatné programy.

PROCES Pr

Spúšťanie

proc_pr

Funkcia

Proces **Pr** pošle signál SIGUSR1 svojmu hlavnému procesu (Zadanie) na signalizáciu, že je pripravený. Proces **Pr** si údaje bude žiadať:

- Ak pošle signál (SIGUSR1) procesu **P1**, nech tento proces (P1) zapíše slovo prečítané zo súboru *p1.txt*.
- To isté platí aj pre proces **P2** a *p2.txt*. Teda, ak proces **Pr** pošle signál (SIGUSR1) procesu **P2** nech do rúry R1 zapíše slovo proces **P2** zo súboru *p2.txt*.

Proces **Pr** k slovu prijatému z rúry R1 pridá svoju značku a zapíše nové slovo do rúry R2.

PROCES S

Spúšťanie

proc_s

Funkcia

Proces **S** pošle signál SIGUSR1 svojmu hlavnému procesu (Zadanie) na signalizáciu, že je (proces S) pripravený. Proces **S** prijme slovo zo zdieľanej pamäte SM1 so synchronizáciou semaforom S1 (pozri časť o semaforoch v časti „Popis komunikácie“), pripíše k nemu svoju značku a zapíše ho do zdieľanej pamäte SM2 so synchronizáciou semaforom S2.

PROCES Serv1

Spúšťanie

proc_serv1 <číslo portu 1> <číslo portu 2>

Funkcia

Proces **Serv1** vytvorí TCP server (na porte <číslo portu 1>), ktorý bude prijímať TCP pakety. Server prijaté slová označí svojou značkou a pošle ich ďalej na UDP server (port <číslo portu 2>). Čísla portov 1 a 2 sú argumenty hlavného procesu (pozri kapitolu „PROCES Zadanie“).

Proces **Serv1** pošle signál SIGUSR1 svojmu hlavnému procesu (Zadanie) na signalizáciu, že je (proces Serv1) pripravený. TCP aj UDP server nech vytvorený na lokálnom počítači, teda na počítači „127.0.0.1“ (pozor, nie „localhost“!).

POPIS KOMUNIKÁCIE

Semafor S1

Pre semafor S1 je potrebné vytvoriť dvojicu semaforov. Proces **T** nech sa riadi podľa semaforu S1[0] a proces **S** sa bude riadiť podľa semaforu S1[1], pričom nastavený semafor S1[0] (rozumej nastavený na hodnotu 1) nech znamená, že proces **T** môže zapisovať do zdieľanej pamäte (SM1). Nastavený semafor S1[1] nech znamená, že proces **S** môže zo zdieľanej pamäte (SM1) údaje čítať.

Semafor S2

Pre semafor S2 je potrebné vytvoriť dvojicu semaforov. Proces **S** nech sa riadi podľa semaforu S2[0] a proces **D** sa bude riadiť podľa semaforu S2[1]. Pričom nastavený semafor S2[0] (rozumej nastavený na hodnotu 1) nech znamená, že proces **S** môže zapisovať do zdieľanej pamäte (SM2). Nastavený semafor S2[1] nech znamená, že proces **D** môže zo zdieľanej pamäte (SM2) údaje čítať.

Príklad komunikácie medzi procesom T a procesom S – semafor S1

Pozn.: Pre semafor S2 je to analogické.

Hodnota semaforu	Význam
0	červená
1	zelená

Proces T sa riadi semaforom S1[0] a proces S sa riadi semaforom S1[1]. To znamená, že kým ma proces T na svojom semafore (S1[0]) hodnotu 0 (t.j. červená) – tak stále čaká. To isté platí pre proces S. To znamená, že kým proces S má na svojom semafore (S1[1]) hodnotu 0 (červená) – to znamená, že musí čakať.

Semafor treba inicializovať do nasledovného stavu:

S1[0] = 1 (zelená/môžeš)

S1[1] = 0 (červená/stoj)

- tento stav znamená, že proces T môže do zdieľanej pamäte zapisovať. Proces S čaká, má červenú, nemôže čítať.

Proces T má zelenú (teda môže zapisovať), lebo má svoj semafor (S1[0]) nastavený na hodnotu 1 (zelená). Teda do zdieľanej pamäte zapíše svoje údaje. Teraz je potrebné, aby proces T umožnil procesu S údaje čítať. Preto zmení semafor na nasledovný stav:

[Najprv zmení svoj semafor (semafor S1[0]) z hodnoty 1 na hodnotu 0 (teda zo zelenej na červenú) a semafor procesu S (teda S1[1]) zmení z hodnoty 0 na hodnotu 1 (z červenej na zelenú, aby mu naznačil, že údaje sú zapísané a môže ich teda čítať.)]

S1[0] = 0 (červená/stoj)

S1[1] = 1 (zelená/môžeš)

- tento stav znamená, že proces T nemôže do zdieľanej pamäte zapisovať, má čakať. Proces S má zelenú, má v zdieľanej pamäti pripravené údaje a môže údaje z pamäte prečítať.

Teraz už môže semafor S zo zdieľanej pamäte údaje prečítať, lebo jeho semafor (S1[1]) sa zmenil z hodnoty 0 (červená) na hodnotu 1 (zelená). Prečíta teda údaje zo zdieľanej pamäte a vymení farby semaforov (t.j. zmení sebe zo zelenej na červenú – z 1 na 0 – a procesu T zmení z červenej na zelenú – z 0 na 1 – aby mohol do pamäte zapisovať).

Súbory p1.txt, p2.txt a serv2.txt

Súbory p1.txt a p2.txt budú obsahovať slová (rozumej reťazce znakov každé v novom riadku), pričom veľkosť slova počas putovania medzi procesmi nepresiahne dĺžku 150 znakov. Súbor serv2.txt nech obsahuje výsledné slová zapísané každé v samostatnom riadku.

Poznámka: Pre účely vývoja programov a testovania je možné programy **Pr**, **S** a **Serv1** stiahnuť zo stránok systému na odovzdávanie zadaní.

Upozornenie:

Pozor, nachádzate sa v paralelnom prostredí! To znamená, že poradie vykonávania procesov nemusí byť rovnaké na rôznych systémoch. Preto dbajte na synchronizáciu a vyhnite sa problémom „predbiehania“ procesov!

„Čo všetko teda potrebujem spraviť??“

1. Potrebujem spraviť programy
 - a. **P1** (zdrojový text: **proc_p1.cpp**, spustiteľný program: **proc_p1**),
 - b. **P2** (zdrojový text: **proc_p2.cpp**, spustiteľný program: **proc_p2**),
 - c. **T** (zdrojový text: **proc_t.cpp**, spustiteľný program: **proc_t**),
 - d. **D** (zdrojový text: **proc_d.cpp**, spustiteľný program: **proc_d**),
 - e. **Serv2** (zdrojový text: **proc_serv2.cpp**, spustiteľný program: **proc_serv2**) a
 - f. **Zadanie** (zdrojový text: **zadanie.cpp**, spustiteľný program: **zadanie**).A takisto súbor **makefile** na skompilovanie všetkých zadaní.
2. Musím dodržať názvy programov a ich zdrojových textov uvedené v zátvorkách! Názvy súborov zdrojových textov nech sú dodržané tiež.
3. Musím dodržať stanovené názvy vstupných súborov (p1.txt, p2.txt) a výstupného súboru (serv2.txt). Dbáť na veľké a malé písmená.
4. Vyhотовené **zdrojové texty** programov spolu so súborom **makefile** (programy Pr, S a Serv1 nie – tie budú pri kontrole dodané) treba zbaliť, najlepšie vo formáte *.tar.gz (napr. zadanie.tar.gz) a odoslať na server.

2. DODEFINOVANIE ZADANIA

Zadanie Unix1 predstavuje komplexnú ukážku medziprocesovej komunikácie v prostredí Unix/Linux. Riešenie pozostáva z viacerých samostatných procesov, ktoré medzi sebou komunikujú pomocou rúr, zdieľanej pamäte, semaforov, signálov a sieťovej komunikácie (TCP a UDP).

Hlavným procesom je program Zadanie, ktorý vytvára všetky prostriedky medziprocesovej komunikácie (IPC), zabezpečuje ich správnu inicializáciu a následne spúšťa jednotlivé procesy pomocou volaní fork a exec. Jednotlivé procesy sú spustené v presne danom poradí a medzi nimi prebieha synchronizácia pomocou signálov.

Cieľom zadania je správne pochopenie spolupráce viacerých procesov v paralelnom prostredí a správne použitie synchronizačných mechanizmov.

3. POPIS RELEVANTNÝCH ŠTRUKTÚR, ALGORITMOV A DÁTOVÝCH TYPOV

V riešení sú použité základné dátové typy jazyka C, najmä typy `int`, `char` a pole znakov typu `char[]`. Pole znakov má maximálnu veľkosť 150 znakov, čo zodpovedá maximálnej dĺžke slova definovanej v zadaní.

Na synchronizáciu procesov sú použité System V semaforey, konkrétne štruktúra `sembuf`, ktorá určuje operácie nad semaforom. Semaforey sú použité na riadenie prístupu k zdieľanej pamäti medzi procesmi.

Na sieťovú komunikáciu sú použité sockety a štruktúra `sockaddr_in`, ktorá obsahuje informácie o type spojenia, porte a IP adrese.

4. POPIS RIEŠENIA A KOMUNIKÁCIE MEDZI PROCESMI

Procesy P1 a P2 načítavajú slová zo vstupných súborov p1.txt a p2.txt. Po prijatí signálu SIGUSR1 prečítajú jedno slovo a zapíšu ho do rúry R1.

Proces Pr číta slová z rúry R1, pridáva k nim svoju značku a zapisuje ich do rúry R2. Proces T číta dáta z rúry R2 a zapisuje ich do zdieľanej pamäte SM1. Prístup k pamäti je synchronizovaný semaforom S1.

Proces S číta dáta zo zdieľanej pamäte SM1, pridáva vlastnú značku a zapisuje ich do zdieľanej pamäte SM2. Tento proces používa semafore S1 a S2.

Proces D číta dáta zo zdieľanej pamäte SM2 a pomocou TCP komunikácie ich posiela procesu Serv1. Proces Serv1 prijaté dáta označí a pošle ich ďalej pomocou UDP protokolu procesu Serv2, ktorý výsledné slová zapisuje do súboru serv2.txt.

5. POUŽITÁ LITERATÚRA A INFORMAČNÉ ZDROJE

Pri riešení zadania boli použité manuálové stránky (man), študijné materiály z predmetu Operačné systémy a systém Sofia.

6. SYSTÉMOVÁ PRÍRUČKA

Každý proces má presne definované argumenty príkazového riadku. Procesy P1 a P2 pracujú so vstupnými súbormi a zapisujú dáta do rúry. Procesy T a S pracujú so zdieľanou pamäťou a semaformi. Proces D zabezpečuje TCP komunikáciu a proces Serv2 zabezpečuje UDP komunikáciu a zápis výstupu.

Proces Zadanie zabezpečuje vytvorenie a korektné uvoľnenie všetkých prostriedkov medziprocesovej komunikácie (IPC).

7. PRÍRUČKA POUŽÍVATEĽA

Účel programu je demonštrovať medziprocesovú komunikáciu v operačnom systéme Unix/Linux.

Pred spustením programu je potrebné vykonať kompiláciu pomocou príkazu make. Po úspešnej kompilácii sa program spúšťa príkazom:

`./zadanie <tcp_port> <udp_port>`

Vstupné súbory p1.txt a p2.txt musia byť umiestnené v rovnakom adresári ako spustiteľné súbory.

8. ZDROJOVÝ TEXT V JAZYKU C

Zadanie.c

```
#include <errno.h>
```

```
#include <signal.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/sem.h>
```

```
#include <sys/shm.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <unistd.h>
```

```
#define WORD_MAX 150
```

```
// len aby pause() prebudil signal
```

```
static void sig_ready(int sig) {
```

```
    (void)sig;
```

```
    // printf("Odchytenie signalu %d\n", sig);
```

```
}
```

```
// pomocna funkcia: vytvori string z int (pre exec argumenty)
```

```
static void int_na_str(int val, char *out, size_t out_sz) {
```

```
    snprintf(out, out_sz, "%d", val);
```

```
}
```

```
int main(int argc, char *argv[]) {
```

```
    // handler na SIGUSR1 - pouzije sa pri "som pripraveny" medzi procesmi
```

```
    (void)signal(SIGUSR1, sig_ready);
```

```
    // kontrola argumentov
```

```
    if (argc != 3) {
```

```
        fprintf(stderr, "Pouzitie: %s <port1> <port2>\n", argv[0]);
```

```
        // v testoch sa aj tak vola so spravnymi argumentmi
```

```
        // toto je len ochrana pred segfaultom
```

```
        return EXIT_FAILURE;
```

```
    }
```

```
    // porty zo zadania
```

```
    char *port_tcp_serv1 = argv[1];
```

```
    char *port_udp_serv2 = argv[2];
```

```
    // RURY (pipe)
```

```
    // R1: P1/P2 -> Pr (zapis) / Pr (citanie)
```

```
    // R2: Pr (zapis) -> T (citanie)
```

```
    int r1_fd[2];
```

```
    int r2_fd[2];
```

```
    pipe(r1_fd);
```

```
    pipe(r2_fd);
```

```
// fd pre exec argumenty (ako string)

char r1_str_zapis[16], r1_str_citanie[16];

char r2_str_zapis[16], r2_str_citanie[16];

int_na_str(r1_fd[1], r1_str_zapis, sizeof(r1_str_zapis));

int_na_str(r1_fd[0], r1_str_citanie, sizeof(r1_str_citanie));

int_na_str(r2_fd[1], r2_str_zapis, sizeof(r2_str_zapis));

int_na_str(r2_fd[0], r2_str_citanie, sizeof(r2_str_citanie));
```

```
// SEMAFORY
```

```
// S1 pre T <-> S (SM1)
```

```
// S2 pre S <-> D (SM2)
```

```
key_t key_s1 = ftok("proc_t", 'X');
```

```
key_t key_s2 = ftok("proc_d", 'X');
```

```
// ziskanie semaforu
```

```
int sem_s1 = semget(key_s1, 2, IPC_CREAT | 0666);
```

```
// kontrola chyby
```

```
if (sem_s1 == -1)
```

```
    perror("zadanie: semget S1");
```

```
// ziskanie semaforu
```

```
int sem_s2 = semget(key_s2, 2, IPC_CREAT | 0666);
```

```
// kontrola chyby
```

```
if (sem_s2 == -1)
```

```
    perror("zadanie: semget S2");
```

```

// inicializacia "semaforoveho semaforu" - farby

// S1: T moze zapisovat (1), S caka (0)

semctl(sem_s1, 0, SETVAL, 1);

semctl(sem_s1, 1, SETVAL, 0);

// S2: S moze zapisovat (1), D caka (0)

semctl(sem_s2, 0, SETVAL, 1);

semctl(sem_s2, 1, SETVAL, 0);


char sem1_str[16], sem2_str[16];

int_na_str(sem_s1, sem1_str, sizeof(sem1_str));

int_na_str(sem_s2, sem2_str, sizeof(sem2_str));


// ZDIELANE PAMATE

int shm_sm1 = shmget(IPC_PRIVATE, WORD_MAX, IPC_CREAT | 0666);

// kontrola chyby

if (shm_sm1 == -1)

    perror("zadanie: shmget SM1");


// ziskanie zdielanej pamate

int shm_sm2 = shmget(IPC_PRIVATE, WORD_MAX, IPC_CREAT | 0666);

// kontrola chyby

if (shm_sm2 == -1)

    perror("zadanie: shmget SM2");


char sm1_str[16], sm2_str[16];

```

```
int_na_str(shm_sm1, sm1_str, sizeof(sm1_str));

int_na_str(shm_sm2, sm2_str, sizeof(sm2_str));


// PID pre procesy

int pid_p1 = -1, pid_p2 = -1, pid_pr = -1;

int pid_t = -1, pid_s = -1, pid_d = -1;

int pid_serv1 = -1, pid_serv2 = -1;


// priprava argv pre exec (p1/p2 pisu do r1)

char *argv_p1[] = {"proc_p1", r1_str_zapis, NULL};

char *argv_p2[] = {"proc_p2", r1_str_zapis, NULL};


// proc_pr (bude dodany pri kontrole)

char pid_p1_str[16], pid_p2_str[16];


// proc_t (cita z r2, zapisuje do sm1, riadi sa s1)

char *argv_t[] = {"proc_t", sem1_str, sm1_str, r2_str_citanie, NULL};


// proc_s (bude dodany pri kontrole)

char *argv_s[] = {"proc_s", sm1_str, sem1_str, sm2_str, sem2_str, NULL};


// proc_d (cita sm2 a posielala do TCP serv1)

char *argv_d[] = {"proc_d", sem2_str, sm2_str, port_tcp_serv1, NULL};


// serv2 (UDP server na porte2)

char *argv_serv2[] = {"proc_serv2", port_udp_serv2, NULL};
```



```
// serv1 (bude dodany pri kontrole)

char *argv_serv1[] = {"proc_serv1", port_tcp_serv1, port_udp_serv2, NULL};

// SPAWN: P1

pid_p1 = fork();

if (pid_p1 == -1) {

    perror("zadanie: fork P1");

    exit(EXIT_FAILURE);

}

if (pid_p1 == 0) {

    execve("proc_p1", argv_p1, NULL);

    _exit(EXIT_FAILURE);

}

pause();

// SPAWN: P2

pid_p2 = fork();

if (pid_p2 == -1) {

    perror("zadanie: fork P2");

    exit(EXIT_FAILURE);

}

if (pid_p2 == 0) {

    execve("proc_p2", argv_p2, NULL);

    _exit(EXIT_FAILURE);

}
```

```
pause();
```

```
// pid pre proc_pr ako string
```

```
int_na_str(pid_p1, pid_p1_str, sizeof(pid_p1_str));
```

```
int_na_str(pid_p2, pid_p2_str, sizeof(pid_p2_str));
```

```
char *argv_pr[] = {"proc_pr", pid_p1_str, pid_p2_str, r1_str_citanie, r2_str_zapis, NULL};
```

```
// SPAWN: Pr
```

```
pid_pr = fork();
```

```
if (pid_pr == -1) {
```

```
    perror("zadanie: fork Pr");
```

```
    exit(EXIT_FAILURE);
```

```
}
```

```
if (pid_pr == 0) {
```

```
    execve("proc_pr", argv_pr, NULL);
```

```
    _exit(EXIT_FAILURE);
```

```
}
```

```
pause();
```

```
// SPAWN: T
```

```
pid_t = fork();
```

```
if (pid_t == -1) {
```

```
    perror("zadanie: fork T");
```

```
    exit(EXIT_FAILURE);
```

```
}
```

```
if (pid_t == 0) {  
    execve("proc_t", argv_t, NULL);  
    _exit(EXIT_FAILURE);  
}  
pause();  
  
// SPAWN: S  
pid_s = fork();  
if (pid_s == -1) {  
    perror("zadanie: fork S");  
    exit(EXIT_FAILURE);  
}  
if (pid_s == 0) {  
    execve("proc_s", argv_s, NULL);  
    _exit(EXIT_FAILURE);  
}  
pause();  
  
// SPAWN: Serv2 (UDP)  
pid_serv2 = fork();  
if (pid_serv2 == -1) {  
    perror("zadanie: fork Serv2");  
    exit(EXIT_FAILURE);  
}  
if (pid_serv2 == 0) {  
    execve("proc_serv2", argv_serv2, NULL);
```

```
    _exit(EXIT_FAILURE);
}

// SPAWN: Serv1 (TCP+UDP forwarder)

pid_serv1 = fork();

if (pid_serv1 == -1) {
    perror("zadanie: fork Serv1");
    exit(EXIT_FAILURE);
}

if (pid_serv1 == 0) {
    execve("proc_serv1", argv_serv1, NULL);
    _exit(EXIT_FAILURE);
}

pause();

// SPAWN: D

pid_d = fork();

if (pid_d == -1) {
    perror("zadanie: fork D");
    exit(EXIT_FAILURE);
}

if (pid_d == 0) {
    execve("proc_d", argv_d, NULL);
    _exit(EXIT_FAILURE);
}

pause();
```

```
// CAKANIE na ukoncenie serv2
```

```
// serv2 skonci sam po prijati 10 slov a zapise ich do serv2.txt
```

```
int status = 0;
```

```
waitpid(pid_serv2, &status, 0);
```

```
// VYPNUTIE VSETKYCH PROCESSOV
```

```
kill(pid_p1, SIGINT);
```

```
kill(pid_p2, SIGINT);
```

```
kill(pid_pr, SIGINT);
```

```
kill(pid_t, SIGINT);
```

```
kill(pid_s, SIGINT);
```

```
kill(pid_d, SIGINT);
```

```
kill(pid_serv1, SIGINT);
```

```
kill(pid_serv2, SIGINT);
```

```
// UPRATANIE IPC ZDROJOV
```

```
if (semctl(sem_s1, 0, IPC_RMID) == -1) {
```

```
    perror("zadanie: semctl remove S1");
```

```
}
```

```
// nastavenie/mazanie semaforu
```

```
if (semctl(sem_s2, 0, IPC_RMID) == -1) {
```

```
    perror("zadanie: semctl remove S2");
```

```
}
```

```
// uprava/mazanie shm
```

```

    if (shmctl(shm_sm1, IPC_RMID, 0) == -1) {
        perror("zadanie: shmctl remove SM1");
    }

    // uprava/mazanie shm

    if (shmctl(shm_sm2, IPC_RMID, 0) == -1) {
        perror("zadanie: shmctl remove SM2");
    }

    return 0;
}

Proc_p1

#include <errno.h>

#include <fcntl.h>

#include <signal.h>

#include <stdio.h>

#include <stdlib.h>

#include <sys/stat.h>

#include <sys/types.h>

#include <unistd.h>

// globalne deskripty, aby k nim mal pristup aj signal handler

static int g_fd_rura_zapis = -1; // zapisovaci koniec rury R1

static int g_fd_subor = -1; // otvoreny subor p1.txt

// ked dostaneme SIGINT, tak sa pekne ukoncime (zadanie nas takto vypina)

static void koniec_na_sigint(int sig) {

```

```

(void)sig;

_exit(EXIT_SUCCESS);
}

// ked dostaneme SIGUSR1, tak posleme dalsie slovo do rury
static void posli_jedno_slovo(int sig) {

    (void)sig;

    // citam znak po znaku a rovno ho posielam do rury
    // skonci to, ked narazim na \n (koniec slova/riadku)
    char znak = 0;
    do {
        // ignorujem navratove hodnoty, lebo pri hodnoteni sa predpoklada korektne vstupy
        read(g_fd_subor, &znak, 1);
        write(g_fd_rura_zapis, &znak, 1);
    } while (znak != '\n');
}

int main(int argc, char *argv[]) {

    // otvorime vstupny subor
    g_fd_subor = open("p1.txt", O_RDONLY);

    // kontrola chyby
    if (g_fd_subor == -1) {

    // otvorenie suboru

        perror("proc_p1: open(p1.txt)");

    }
}

```

```
// argument je fd zapisovacieho konca rury R1
```

```
if (argc >= 2) {
```

```
    g_fd_rura_zapis = atoi(argv[1]);
```

```
}
```

```
// nastavime reakcie na signaly
```

```
(void)signal(SIGUSR1, posli_jedno_slovo);
```

```
(void)signal(SIGINT, koniec_na_sigint);
```

```
// poviem parentovi, ze som pripraveny (zadanie potom spravi pause())
```

```
kill(getppid(), SIGUSR1);
```

```
// nic nerobim, iba cakam na signaly
```

```
while (1) {
```

```
    sleep(1);
```

```
}
```

```
return 0;
```

```
}
```

```
Proc_p2
```

```
#include <errno.h>
```

```
#include <fcntl.h>
```

```
#include <signal.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```



```
#include <sys/stat.h>

#include <sys/types.h>

#include <unistd.h>


// globalne deskripty (signal handler ich potrebuje)

static int g_fd_rura_zapis = -1; // zapisovaci koniec rury R1

static int g_fd_subor = -1; // otvoreny subor p2.txt


// ukoncenie na SIGINT (zadanie to pouzije na vypnutie)

static void koniec_na_sigint(int sig) {

    (void)sig;

    _exit(EXIT_SUCCESS);

}


// poslanie jedneho slova po SIGUSR1

static void posli_jedno_slovo(int sig) {

    (void)sig;


    char znak = 0;

    do {

        // citanie dat

        read(g_fd_subor, &znak, 1);

        write(g_fd_rura_zapis, &znak, 1);

    } while (znak != '\n');

}
```

```
int main(int argc, char *argv[]) {  
  
    // otvorime p2.txt  
    g_fd_subor = open("p2.txt", O_RDONLY);  
  
    // kontrola chyby  
    if (g_fd_subor == -1) {  
  
        // otvorenie suboru  
  
        perror("proc_p2: open(p2.txt)");  
  
    }  
  
  
    // z argv berieme fd zapisovacieho konca rury  
    if (argc >= 2) {  
  
        g_fd_rura_zapis = atoi(argv[1]);  
  
    }  
  
  
    // nastavime signal handlery  
  
    (void)signal(SIGUSR1, posli_jedno_slovo);  
  
    (void)signal(SIGINT, koniec_na_sigint);  
  
  
    // oznamime parentovi, ze sme pripraveny  
    kill(getppid(), SIGUSR1);  
  
  
    // cakanie na signaly  
    while (1) {  
  
        sleep(1);  
  
    }  
}
```

```
    return 0;
}

Proc_d

#include <arpa/inet.h>

#include <netdb.h>

#include <netinet/in.h>

#include <signal.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <sys/ipc.h>

#include <sys/sem.h>

#include <sys/shm.h>

#include <sys/socket.h>

#include <sys/types.h>

#include <unistd.h>


#define WORD_MAX 150


static int g_sem_s2 = -1;


// ukoncime sa na SIGINT (zadanie nas killne)

static void koniec_na_sigint(int sig) {

    (void)sig;

    _exit(EXIT_SUCCESS);

}
```

```
// D caka na S2[1] (moje povolenie citat)
```

```
static int sem_p_d(void) {
```

```
    struct sembuf op;
```

```
    op.sem_num = 1; // D sa riadi semaforom S2[1]
```

```
    op.sem_op = -1;
```

```
    op.sem_flg = SEM_UNDO;
```

```
// operacia so semaforom
```

```
    if (semop(g_sem_s2, &op, 1) == -1) {
```

```
        fprintf(stderr, "proc_d: semop P zlyhal\n");
```

```
        return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
// po precitani dat pusti S, aby mohol zas zapisovat do SM2 (S2[0])
```

```
static int sem_v_pre_s(void) {
```

```
    struct sembuf op;
```

```
    op.sem_num = 0; // pre S je to S2[0]
```

```
    op.sem_op = 1;
```

```
    op.sem_flg = SEM_UNDO;
```

```
// operacia so semaforom
```

```
    if (semop(g_sem_s2, &op, 1) == -1) {
```

```
        fprintf(stderr, "proc_d: semop V zlyhal\n");
```

```
        return 0;

    }

    return 1;
}
```

```
// vytvori TCP spojenie na 127.0.0.1:<port>
```

```
static int pripoj_sa_na_serv1(int port) {

    int sock = socket(PF_INET, SOCK_STREAM, 0);

    // kontrola chyby

    if (sock < 0) {

        perror("proc_d: socket");

        return -1;

    }

}
```

```
// podľa zadania ma byť 127.0.0.1 (nie localhost)
```

```
struct hostent *server = gethostbyname("127.0.0.1");
```

```
if (server == NULL) {

    perror("proc_d: gethostbyname");

}
```

```
// zavretie suboru/socketu
```

```
    close(sock);

    return -1;

}
```

```
struct sockaddr_in serv_addr;
```

```
memset(&serv_addr, 0, sizeof(serv_addr));
```

```
serv_addr.sin_family = AF_INET;

memcpy(&serv_addr.sin_addr.s_addr, server->h_addr, (size_t)server->h_length);

serv_addr.sin_port = htons((uint16_t)port);
```

```
// pripojenie na server
```

```
if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {

    perror("proc_d: connect");
```

```
// zavretie suboru/socketu
```

```
    close(sock);

    return -1;
```

```
}
```

```
return sock;
```

```
}
```

```
int main(int argc, char *argv[]) {
```

```
    if (argc < 4) {

        fprintf(stderr, "proc_d: malo argumentov\n");

        return EXIT_FAILURE;
```

```
    }
```

```
    g_sem_s2 = atoi(argv[1]);
```

```
    int id_sm2 = atoi(argv[2]);
```

```
    int port_serv1 = atoi(argv[3]);
```

```
// pripojime zdielanu pamat SM2
```

```
void *sm2_ptr = shmat(id_sm2, (void *)0, 0);

if (sm2_ptr == (void *)-1) {
// pripojenie zdieľanej pamäte

    perror("proc_d: shmat(SM2)");
}


// TCP spojenie na Serv1

int sockfd = pripoj_sa_na_serv1(port_serv1);
// kontrola chyby

if (sockfd < 0) {

    return EXIT_FAILURE;
}


// signal handlery

(void)signal(SIGINT, koniec_na_sigint);


// oznamime parentovi, ze sme pripraveny

kill(getppid(), SIGUSR1);


// hlavna slučka: precítam zo SM2 a poslem do TCP

while (1) {

    // čakam kým S zapíše a pusti ma citat

    if (!sem_p_d()) {

        exit(EXIT_FAILURE);
    }
}
```

```

// poslem cely blok (150B), tak ako to bolo v povodnom rieseni

// Serv1 ocakava retazec, ale posielame fixnu dlzku.
(void)write(sockfd, sm2_ptr, WORD_MAX);


// pustim S aby mohol zapisovat dalsie slovo
if (!sem_v_pre_s()) {
    exit(EXIT_FAILURE);
}


// vycistim SM2 (nie je to nutne, ale je to prehľadne)
memset(sm2_ptr, 0, WORD_MAX);


// maly sleep, aby to neletelo brutalne rychlo
sleep(1);
}


return 0;
}

Proc_t

#include <errno.h>

#include <signal.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <sys/ipc.h>

#include <sys/sem.h>

```



```

#include <sys/shm.h>

#include <sys/types.h>

#include <unistd.h>


#define WORD_MAX 150


// globalny semafor, aby ho videli aj pomocne funkcie

static int g_sem_s1 = -1;


// pri ukonceni procesu (SIGINT) chceme iba rychlo skoncit

static void koniec_na_sigint(int sig) {

    (void)sig;

    _exit(EXIT_SUCCESS);

}


// P operacia pre semafor (T caka na svoju "zelenu" = index 0)

static int sem_p_t(void) {

    struct sembuf op;

    op.sem_num = 0;    // T pouziva S1[0]

    op.sem_op = -1;    // zober jednu "token" (cakaj ak je 0)

    op.sem_flg = SEM_UNDO;


// operacia so semaforom

    if (semop(g_sem_s1, &op, 1) == -1) {

        fprintf(stderr, "proc_t: semop P zlyhal\n");

        return 0;
    }
}

```

```
}  
  
return 1;  
  
}
```

```
// V operacia (T po zapise rozsvieti zelenu pre S = index 1)
```

```
static int sem_v_pre_s(void) {
```

```
    struct sembuf op;
```

```
    op.sem_num = 1;    // S dostane S1[1]
```

```
    op.sem_op = 1;    // pridaj token
```

```
    op.sem_flg = SEM_UNDO;
```

```
// operacia so semaforom
```

```
    if (semop(g_sem_s1, &op, 1) == -1) {
```

```
        fprintf(stderr, "proc_t: semop V zlyhal\n");
```

```
        return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
// nacita jeden riadok z FD (po \n), ulozi do out (nul-terminated)
```

```
static int nacitaj_riadok(int fd, char *out, size_t out_sz) {
```

```
    // citame po znakoch, lebo format zadania je "slovo\n"
```

```
    size_t idx = 0;
```

```
    while (idx + 1 < out_sz) {
```

```
        char ch = 0;
```

```
    // citanie dat
```

```

    ssize_t r = read(fd, &ch, 1);

    if (r <= 0) {

        // EOF alebo chyba

        return 0;

    }

    if (ch == '\n') {

        out[idx] = '\0';

        return 1;

    }

    out[idx++] = ch;

}

// ak sa slovo zmestilo presne, ukoncime retazec

out[out_sz - 1] = '\0';

return 1;

}

int main(int argc, char *argv[]) {

    // argumenty: semafor, shm, pipe

    if (argc < 4) {

        fprintf(stderr, "proc_t: malo argumentov\n");

        return EXIT_FAILURE;

    }

```

```
g_sem_s1 = atoi(argv[1]);

int id_sm1 = atoi(argv[2]);

int fd_r2_citanie = atoi(argv[3]);


// pripojime zdielanu pamat
void *sm1_ptr = shmat(id_sm1, (void *)0, 0);

if (sm1_ptr == (void *)-1) {
// pripojenie zdielanej pamate
    perror("proc_t: shmat(SM1)");
}


// nastavenie signalov
(void)signal(SIGINT, koniec_na_sigint);


// poviem parentovi, ze som pripraveny
kill(getppid(), SIGUSR1);


// buffer na jedno slovo
char *slovo = (char *)calloc(1, WORD_MAX);

// kontrola podmienky
if (!slovo) {
    perror("proc_t: calloc");
    return EXIT_FAILURE;
}


// hlavna slucka
```

```
while (1) {  
  
    // cakam kym mam povolene zapisovat do SM1  
  
    if (!sem_p_t()) {  
  
        exit(EXIT_FAILURE);  
  
    }  
  
  
    // pripravim si miesto v pamati aj v buffri  
  
    memset(sm1_ptr, 0, WORD_MAX);  
  
    memset(slovo, 0, WORD_MAX);  
  
  
    // nacitam riadok z rury  
  
    if (!nacitaj_riadok(fd_r2_citanie, slovo, WORD_MAX)) {  
  
        // ak by pipe skončila, tak tu len ticho koncime  
  
        exit(EXIT_SUCCESS);  
  
    }  
  
  
    // zapis do SM1 (retazec je uz ukonceny \0)  
  
    strcpy((char *)sm1_ptr, slovo);  
  
  
    // pustim proces S nech moze citat  
  
    if (!sem_v_pre_s()) {  
  
        exit(EXIT_FAILURE);  
  
    }  
  
}  
  
return 0;
```

```
}
```

```
Proc_serv2
```

```
#include <netinet/in.h>
```

```
#include <signal.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <sys/socket.h>
```

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
#define WORD_MAX 150
```

```
#define POCET_SLOV 10
```

```
static void koniec_na_sigint(int sig) {
```

```
    (void)sig;
```

```
    _exit(EXIT_SUCCESS);
```

```
}
```

```
int main(int argc, char *argv[]) {
```

```
    if (argc < 2) {
```

```
        fprintf(stderr, "proc_serv2: malo argumentov\n");
```

```
        return EXIT_FAILURE;
```

```
    }
```

```
int port_udp = atoi(argv[1]);
```

```
// vytvorim UDP socket
```

```
int fd_sock = socket(PF_INET, SOCK_DGRAM, 0);
```

```
// kontrola chyby
```

```
if (fd_sock < 0) {
```

```
    perror("proc_serv2: socket");
```

```
}
```

```
// bind na zvoleny port
```

```
struct sockaddr_in addr_srv;
```

```
memset(&addr_srv, 0, sizeof(addr_srv));
```

```
addr_srv.sin_family = AF_INET;
```

```
addr_srv.sin_addr.s_addr = INADDR_ANY;
```

```
addr_srv.sin_port = htons((uint16_t)port_udp);
```

```
// bind na port/adresu
```

```
if (bind(fd_sock, (struct sockaddr *)&addr_srv, sizeof(addr_srv)) < 0) {
```

```
    perror("proc_serv2: bind");
```

```
}
```

```
// signal handlery
```

```
(void)signal(SIGINT, koniec_na_sigint);
```

```
// oznamim parentovi, ze som pripraveny
```

```
kill(getppid(), SIGUSR1);
```

```
// otvorim vystupny subor (prepisem ho)
```

```
int fd_out = open("serv2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0666);
```

```
// kontrola chyby
```

```
if (fd_out == -1) {
```

```
    // otvorenie suboru
```

```
    perror("proc_serv2: open(serv2.txt)");
```

```
}
```

```
char buf[WORD_MAX];
```

```
memset(buf, 0, sizeof(buf));
```

```
// prijmem presne POCET_SLOV sprav a kazdu zapis... => test tak ockava
```

```
for (int i = 0; i < POCET_SLOV; i++) {
```

```
    memset(buf, 0, sizeof(buf));
```

```
    // prijme data zo socketu a ulozi ich do buffra
```

```
    recv(fd_sock, buf, WORD_MAX, 0);
```

```
    // do suboru zapisujem iba realnu dlzku retazca
```

```
    write(fd_out, buf, (size_t)strlen(buf));
```

```
    write(fd_out, "\n", 1);
```

```
}
```

```
// zavretie suboru/socketu
```

```
close(fd_out);
```



```
close(fd_sock);
```

```
return 0;
```

```
}
```